

<i>Trường ĐH Công Thương TP.HCM</i> Khoa: CNTT Bộ môn: Kỹ thuật phần mềm LẬP TRÌNH MÃ NGUỒN MỞ	BÀI 7 LARAVEL FRAMEWORK Routing nâng cao, Controller RESTful, Blade nâng cao và Form cơ bản	
---	---	---

A. MỤC TIÊU

- Vận dụng Routing nâng cao: tham số động/tùy chọn, ràng buộc regex, named route, route group (prefix/middleware).
- Xây dựng Controller RESTful và dùng resource routes.
- Tổ chức giao diện với Blade Layout, @include partials, @stack/@push, và Blade Components (chuẩn <x-...>).
- Tạo Form cơ bản có CSRF, method spoofing (PUT/PATCH/DELETE), validation inline và hiển thị lỗi.

B. NỘI DUNG THỰC HÀNH

1. Cơ sở lý thuyết

- Routing (*web.php*): `Route::get/post/put/delete`, tham số {id}, tham số tùy chọn {slug?}, ràng buộc `where()`, named route `.name('route.tên')`, group với `prefix()` và `middleware()`.
- Controller RESTful: `php artisan make:controller ArticleController --resource` sinh sẵn 7 action: `index/create/store/show/edit/update/destroy`.
- Resource Routes: `Route::resource('articles', ArticleController::class);`
- Blade:
 - Layout: `@extends / @section / @yield`.
 - Partial: `@include('partials.nav')`.
 - Stacks: `@push('scripts') ... @stack('scripts')`.
 - Components: `<x-alert/>`, `<x-input/>` thay thế `@component`.
- Form + CSRF + Validation:
 - Form: `@csrf, @method('PUT')`.
 - Validation inline: `$request->validate([...])`.
 - Hiển thị lỗi: `@error('field') {{ $message }} @enderror`.

2. Bài tập tại lớp

Bài tập 01: Routing nâng cao cho mô-đun bài viết (Articles)

Yêu cầu:

- Tạo các route minh họa: tham số động {page}, tham số tùy chọn {slug?} với regex, và group route có prefix admin.
- Đặt tên (named route) để gọi trong view/controller.
- Sinh viên phải kiểm tra và minh họa cách truy cập route bằng cả URL trực tiếp và hàm route().

Mục tiêu:

- Hiểu cách định nghĩa route nâng cao.
- Nắm kỹ năng dùng named route và group.

Hướng dẫn:

- Mở *routes/web.php* và thêm:

```
use Illuminate\Support\Facades\Route;

// 1. Route có tham số động
Route::get('/articles/page/{page}', function ($page) {
    return "Trang bài viết số: " . (int)$page;
})->whereNumber('page')->name('articles.page');

// 2. Tham số tùy chọn + regex slug
Route::get('/articles/slug/{slug?}', function ($slug = 'khong-co-slug') {
    return "Slug: " . $slug;
})->where('slug', '[a-z0-9-]+');

// 3. Route group với prefix
Route::prefix('admin')->group(function () {
    Route::get('/articles', fn() => 'Quản trị bài viết')
        ->name('admin.articles.index');
});
```

- Kiểm tra nhanh:
 - /articles/page/3 → “Trang bài viết số: 3”
 - /articles/slug/laravel-12-routing → in slug hợp lệ
 - /admin/articles → “Quản trị bài viết”
 - Dùng named route trong view hoặc controller: route('articles.page', ['page'=>2])

Lỗi thường gặp: Regex không khớp → 404; chưa use Route.

Bài tập 02: Controller RESTful & Resource Routes cho Article

Yêu cầu:

- Tạo ArticleController với 7 action chuẩn RESTful.
- Định nghĩa Route::resource('articles', ArticleController::class);.
- Cài đặt logic tối thiểu cho index, create, store, show, edit, update, destroy.
- Với store và update: thêm validate dữ liệu (title, body).
- Kết quả phải có thể: xem danh sách, tạo mới, chỉnh sửa, xóa (demo).

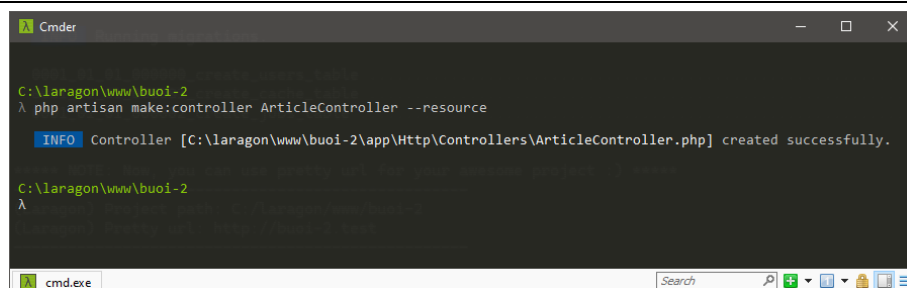
Mục tiêu đạt được:

- Làm quen với resource controller trong Laravel.
- Biết cách tổ chức CRUD theo RESTful..

Hướng dẫn:

- Tạo controller RESTful:

```
php artisan make:controller ArticleController --resource
```



Hình 1 Kết quả tạo controller

- Khai báo resource route trong *routes/web.php*:

```
use App\Http\Controllers\ArticleController;  
Route::resource('articles', ArticleController::class);
```

- Mở *app/Http/Controllers/ArticleController.php* và bổ sung logic tối thiểu:

```
<?php  
  
namespace App\Http\Controllers;  
  
use Illuminate\Http\Request;  
  
class ArticleController extends Controller  
{  
    /**  
     * Display a listing of the resource.  
     */
```

```

public function index()
{
    // Tạm thời dùng mảng mô phỏng dữ liệu
    $articles = [
        ['id' => 1, 'title' => 'Giới thiệu Laravel 12', 'body' => 'Nội
dung A'],
        ['id' => 2, 'title' => 'Blade Components', 'body' => 'Nội dung
B'],
    ];
    return view('articles.index', compact('articles'));
}

/**
 * Show the form for creating a new resource.
 */
public function create()
{
    return view('articles.create');
}

/**
 * Store a newly created resource in storage.
 */
public function store(Request $request)
{
    $validated = $request->validate([
        'title' => ['required', 'string', 'max:255'],
        'body' => ['required', 'string', 'min:10'],
    ]);
    // Tạm thời: giả lưu, thực tế sẽ lưu DB ở buổi sau
    return redirect()->route('articles.index')
        ->with('success', 'Tạo bài viết thành công (demo).');
}

/**
 * Display the specified resource.
 */
public function show(string $id)
{
    return "Xem chi tiết bài viết ID: " . (int)$id;
}

/**

```

```

    * Show the form for editing the specified resource.
    */
    public function edit(string $id)
    {
        // Tạm dữ liệu mẫu
        $article = ['id' => $id, 'title' => 'Tiêu đề mẫu', 'body' => 'Nội
dung mẫu'];
        return view('articles.edit', compact('article'));
    }

    /**
     * Update the specified resource in storage.
     */
    public function update(Request $request, string $id)
    {
        $validated = $request->validate([
            'title' => ['required', 'string', 'max:255'],
            'body' => ['required', 'string', 'min:10'],
        ]);
        return redirect()->route('articles.index')
            ->with('success', "Cập nhật bài viết #$id thành công (demo).");
    }

    /**
     * Remove the specified resource from storage.
     */
    public function destroy(string $id)
    {
        return redirect()->route('articles.index')
            ->with('success', "Đã xóa bài viết #$id (demo).");
    }
}

```

Ghi chú: Ở buổi 3–4, ta sẽ gắn thật với Model Eloquent + Migration.

Bài tập 03: Blade Layout, Partial và Stacks

Yêu cầu:

- Tạo layout chung chứa header, footer.
- Tách navigation và footer thành partials (partials.nav, partials.footer).
- Dùng @stack để thêm scripts riêng ở từng view.
- Xây dựng trang danh sách articles kế thừa layout, hiển thị bảng dữ liệu mẫu.

Mục tiêu đạt được:

- Biết cách tái sử dụng giao diện bằng layout/partial.
- Biết cách chèn script/style theo từng trang bằng stack.

Hướng dẫn:

- Tạo layout: *resources/views/layouts/app.blade.php*

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="utf-8">
  <title>@yield('title','Articles')</title>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <style>
    body { font-family: system-ui, -apple-system, Segoe UI, Roboto, Arial,
sans-serif; }
    .container { max-width: 960px; margin: 24px auto; }
    .flash { padding: 10px; margin-bottom: 12px; background: #ECDF5;
color:#065F46; border-radius:8px; }
    nav a { margin-right: 8px; }
    table { border-collapse: collapse; width: 100%; }
    th, td { border: 1px solid #e5e7eb; padding: 8px; text-align: left; }
    th { background: #f3f4f6; }
  </style>
  @stack('styles')
</head>
<body>
  @include('partials.nav')

  <div class="container">
    @if(session('success'))
      <div class="flash">{{ session('success') }}</div>
    @endif

    @yield('content')
  </div>

  @include('partials.footer')

  @stack('scripts')
</body>
</html>
```

- Tạo *resources/views/partials/nav.blade.php*

```
<nav style="padding:12px;background:#111827;color:white">
```

```

<a href="{{ url('/') }}" style="color:#fff">Trang chủ</a>
<a href="{{ route('articles.index') }}" style="color:#fff">Articles</a>
<a href="{{ route('articles.create') }}" style="color:#fff">Tạo bài
viết</a>
</nav>

```

– Tạo *resources/views/partials/footer.blade.php*

```

<footer style="margin-top:24px;color:#6b7280">
  <hr>
  <small>&copy; HUIT – Khoa CNTT. Laravel 12 Lab.</small>
</footer>

```

– Tạo trang danh sách: *resources/views/articles/index.blade.php*

```

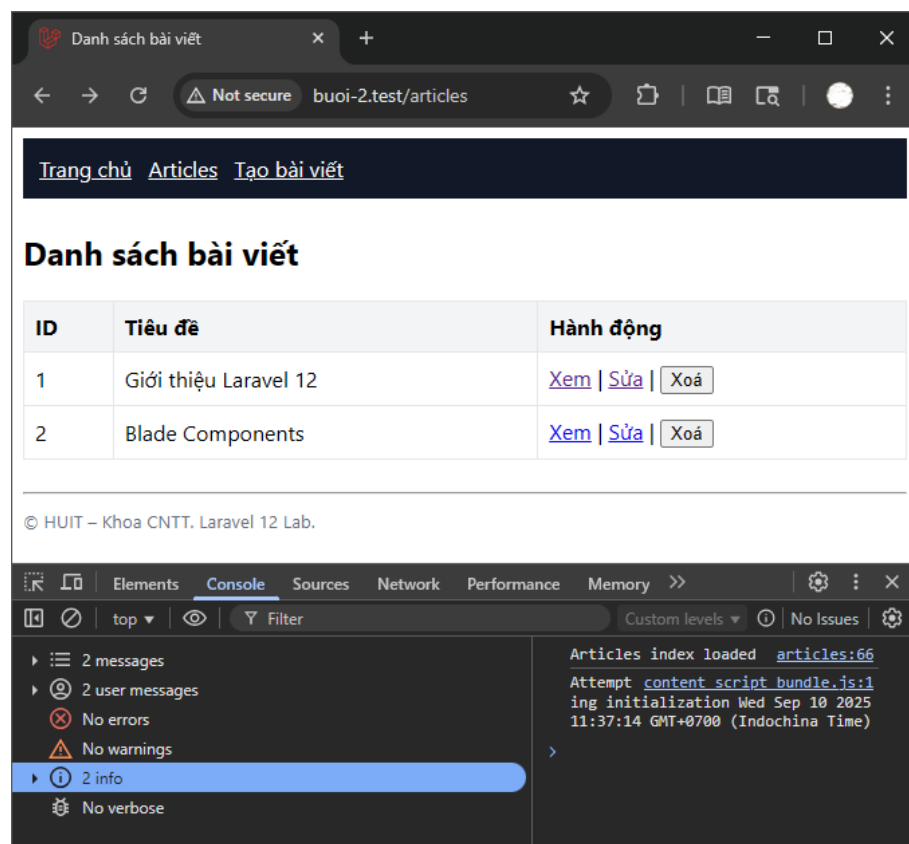
@extends('layouts.app')
@section('title','Danh sách bài viết')

@section('content')
<h2>Danh sách bài viết</h2>
<table>
  <thead>
    <tr><th>ID</th><th>Tiêu đề</th><th>Hành động</th></tr>
  </thead>
  <tbody>
    @forelse($articles as $a)
      <tr>
        <td>{{ $a['id'] }}</td>
        <td>{{ $a['title'] }}</td>
        <td>
          <a href="{{ route('articles.show',$a['id']) }}">Xem</a> |
          <a href="{{ route('articles.edit',$a['id']) }}">Sửa</a> |
          <form action="{{ route('articles.destroy',$a['id']) }}"
method="post" style="display:inline">
            @csrf
            @method('DELETE')
            <button type="submit" onclick="return
confirm('Xóa?')">Xóa</button>
          </form>
        </td>
      </tr>
    @empty
      <tr><td colspan="3">Chưa có bài viết.</td></tr>
    @endforelse
  </tbody>
</table>

```

```
@push('scripts')
<script>
  // demo stack scripts
  console.log('Articles index loaded');
</script>
@endpush
@endsection
```

- Kiểm tra: Mở /articles thấy danh sách, header/footer hiển thị từ partials; console log xuất hiện.



Hình 2 Kết quả bài 03

Bài tập 04: Blade Components: <x-alert> và <x-input>

Yêu cầu:

- Tạo component Alert (hiển thị thông báo).
- Tạo component Input (input text + hiển thị lỗi).
- Áp dụng vào form tạo/sửa bài viết.

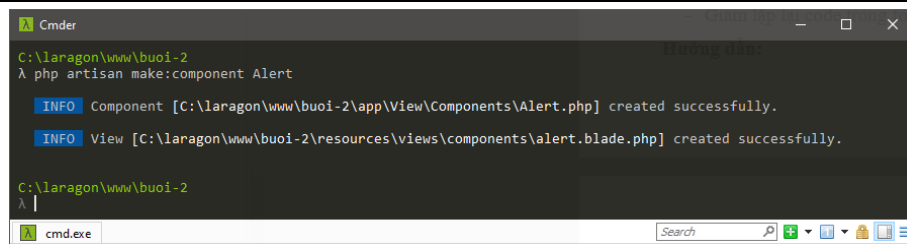
Mục tiêu đạt được:

- Biết tạo và tái sử dụng component Blade.
- Giám lặp lại code trong form.

Hướng dẫn:

- Tạo component Alert:

```
php artisan make:component Alert
```



Hình 3 Kết quả tạo component

- Sửa *resources/views/components/alert.blade.php*:

```
@props(['type' => 'success', 'title' => 'Thông báo'])
<div style="padding:10px;border-radius:8px;margin-bottom:10px;
    background: {{ $type==='success' ? '#ECFDF5' : '#FEF3C7' }};
    color: {{ $type==='success' ? '#065F46' : '#92400E' }};">
    <strong>{{ $title }}:</strong> {{ $slot }}
</div>
```

- Tạo Anonymous component Input: *resources/views/components/input.blade.php*

```
@props(['name', 'label'=>null, 'type'=>'text', 'value'=>''])

<label style="display:block;margin:8px 0 4px">{{ $label ?? ucfirst($name)
}}</label>
<input type="{{ $type }}" name="{{ $name }}" value="{{ old($name, $value) }}"
    style="width:100%;padding:8px;border:1px solid #e5e7eb;border-
radius:6px">

@error($name)
    <div style="color:#991B1B;margin-top:4px">{{ $message }}</div>
@enderror
```

Bây giờ có thể `<x-alert>` và `<x-input>` ở mọi view.

Bài tập 05: Form cơ bản: Tạo/Sửa bài viết + CSRF + Validation inline

Yêu cầu:

- Xây dựng form tạo bài viết (`/articles/create`) với input title, body.
- Bảo vệ CSRF bằng `@csrf`.
- Validate: title required, max 255; body required, min 10 ký tự.
- Hiển thị lỗi ngay dưới trường bằng `@error`.
- Với edit form: dùng `@method('PUT')` để giả lập PUT.
- Sau khi lưu thành công: redirect kèm flash message.

Mục tiêu đạt được:

- Nắm cách xây dựng form cơ bản trong Laravel.
- Hiểu và thực hành CSRF, method spoofing, validation inline.

Hướng dẫn:

- Tạo view tạo mới: *resources/views/articles/create.blade.php*

```
@extends('layouts.app')
@section('title','Tạo bài viết')

@section('content')
<h2>Tạo bài viết</h2>

<x-alert type="warning" title="Lưu ý">Dữ liệu hiện chỉ mô phỏng (chưa lưu DB).</x-alert>

<form action="{{ route('articles.store') }}" method="post">
    @csrf
    <x-input name="title" label="Tiêu đề" />
    <label style="display:block;margin:8px 0 4px">Nội dung</label>
    <textarea name="body" rows="6" style="width:100%;padding:8px;border:1px solid #e5e7eb;border-radius:6px">{{ old('body') }}</textarea>
    @error('body')
        <div style="color:#991b1b;margin-top:4px">{{ $message }}</div>
    @enderror

    <button type="submit" style="margin-top:10px">Lưu</button>
</form>
@endsection
```

- Tạo view chỉnh sửa: *resources/views/articles/edit.blade.php*

```
@extends('layouts.app')
@section('title','Sửa bài viết')

@section('content')
<h2>Sửa bài viết #{{ $article['id'] }}</h2>

<form action="{{ route('articles.update',$article['id']) }}" method="post">
    @csrf
    @method('PUT')

    <x-input name="title" label="Tiêu đề" :value="$article['title']" />
    <label style="display:block;margin:8px 0 4px">Nội dung</label>
```

```

<textarea name="body" rows="6" style="width:100%;padding:8px;border:1px
solid #e5e7eb;border-radius:6px">{{ old('body',$article['body'])
}}</textarea>
@error('body')
    <div style="color:#991B1B;margin-top:4px">{{ $message }}</div>
@enderror

<button type="submit" style="margin-top:10px">Cập nhật</button>
</form>
@endsection

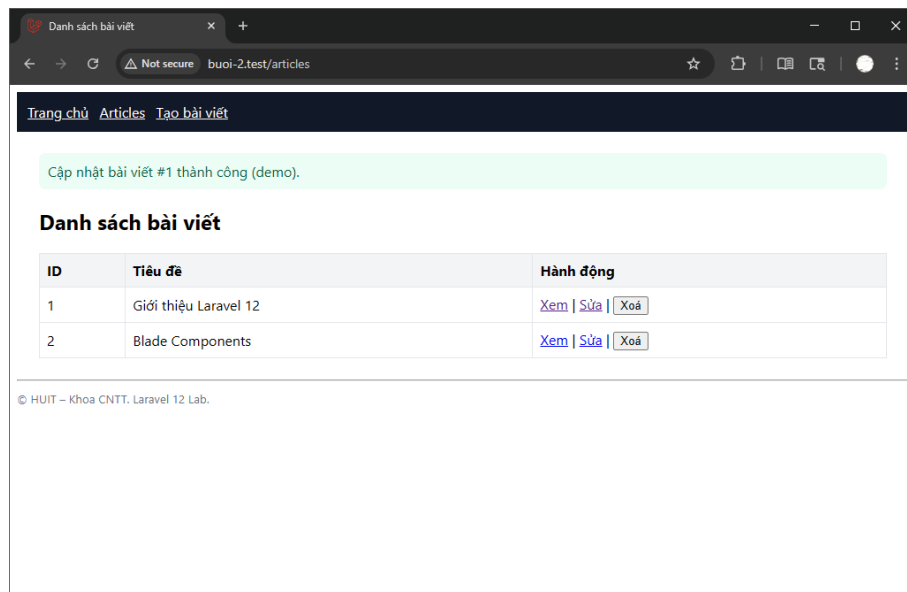
```

– Kiểm tra luồng:

- Mở /articles/create → nhập sai/thiếu → thấy lỗi @error.
- Nhập đúng → redirect /articles + flash “Tạo bài viết thành công (demo).”
- Mở /articles/1/edit → sửa → redirect kèm flash “Cập nhật...”.

The screenshot shows a web browser window with the address bar displaying 'bui-2.test/articles/create'. The page has a dark navigation bar with links 'Trang chủ', 'Articles', and 'Tạo bài viết'. The main heading is 'Tạo bài viết'. Below it is a yellow banner with the text 'Lưu ý: Dữ liệu hiện chỉ mô phỏng (chưa lưu DB)'. The form consists of two main parts: a 'Tiêu đề' (Title) field and a 'Nội dung' (Content) text area. The title field is empty, and a red error message 'The title field is required.' is displayed below it. The content text area contains the text 'test', and a red error message 'The body field must be at least 10 characters.' is displayed below it. At the bottom of the form is a 'Lưu' (Save) button. The footer of the page reads '© HUIT – Khoa CNTT. Laravel 12 Lab.'

Hình 4 Kết quả bài tập 05 - Thêm mới lỗi



Hình 5 Kết quả bài tập 05 - Cập nhật thành công

Lỗi thường gặp: quên `@csrf` → 419; quên `@method('PUT')` khi update.

3. Bài tập ôn luyện

Bài tập 06: Route Model Binding (implicit).

- **Yêu cầu:** Tạo Model Article (chưa cần migration) để demo binding:
 - Route: `/articles/show/{article}` → Article `$article`.
 - Tạm thời mock `findOrFail` (hoặc ở buổi sau gắn CSDL).

Bài tập 07: Form xóa an toàn bằng Confirm & Method Spoofing.

- **Yêu cầu:**
 - Tại `/articles`, thêm nút Xóa mỗi dòng: `<form method="post">@csrf @method('DELETE')</form> + confirm()` JS.
 - Kiểm tra flash sau khi xóa.

Bài tập 08: Nâng cấp UI.

- **Yêu cầu:**
 - Tạo `<x-button>` (anonymous component) với biến thể `primary|danger`.
 - Dùng `@push('styles')` chèn CSS nho nhỏ chỉ ở trang tạo/sửa bài viết.

Bài tập 09: Tối ưu named routes trong view.

- **Yêu cầu:**
 - Dùng `route('articles.create')`, `route('articles.index')` thay vì hard-code.
 - Tạo helper nhỏ (Blade include) hiển thị breadcrumb theo route hiện tại.